

Estructuras de datos

Santiago Gallego

2026-05-07

Contenido

0.1	Ejercicio 1	2
0.1.1	Python	2
0.1.2	R	3
0.1.3	Python	3
0.1.4	R	3
0.1.5	Python	4
0.1.6	R	4
0.1.7	Python	4
0.1.8	R	4
0.1.9	Python	4
0.1.10	R	4
0.1.11	Python	4
0.1.12	R	5
0.2	Ejercicio 2	5
0.2.1	Python	5
0.2.2	R	5
0.2.3	Python	6
0.2.4	R	6
0.2.5	Python	6
0.2.6	R	7
0.2.7	Python	7
0.2.8	R	7
0.2.9	Python	7
0.2.10	R	7
0.3	Pregunta	8

0.1 Ejercicio 1

1. Define tres variables (punto_1, punto_2, punto_3) que almacenen las coordenadas (Latitud, Longitud) como Tuplas estáticas (o su equivalente más seguro en tu lenguaje). Punto 1: 11.1198, -74.0321 Punto 2: 11.1250, -74.0280 Punto 3: 11.1302, -74.0215

0.1.1 Python

```
punto_1 = (11.1198, -74.0321) # Define una tupla inmutable para el primer punto
punto_2 = (11.1250, -74.0280) # Define una tupla inmutable para el segundo punto
punto_3 = (11.1302, -74.0215) # Define una tupla inmutable para el tercer punto
```

0.1.2 R

```
punto_1 <- c(11.1198, -74.0321) # Crea un vector numérico con las coordenadas del punto 1
punto_2 <- c(11.1250, -74.0280) # Crea un vector numérico con las coordenadas del punto 2
punto_3 <- c(11.1302, -74.0215) # Crea un vector numérico con las coordenadas del punto 3
```

2. Crea una colección secuencial y mutable (Lista en Python/Julia o Vector/Lista en R) llamada `ruta_avistamiento` que contenga los tres puntos iniciales en orden.

0.1.3 Python

```
ruta_avistamiento = [ # Inicia una lista mutable
    punto_1,          # Agrega el punto 1 a la lista
    punto_2,          # Agrega el punto 2 a la lista
    punto_3           # Agrega el punto 3 a la lista
]

print("Ruta inicial:", ruta_avistamiento) # Muestra el contenido de la lista en consola
```

Ruta inicial: [(11.1198, -74.0321), (11.125, -74.028), (11.1302, -74.0215)]

0.1.4 R

```
ruta_avistamiento <- list(punto_1, punto_2, punto_3) # Agrupa los vectores en una lista de R
cat("Ruta inicial:\n")                               # Imprime un encabezado de texto
```

Ruta inicial:

```
print(ruta_avistamiento)                             # Muestra la estructura de la lista
```

```
[[1]]
[1] 11.1198 -74.0321
```

```
[[2]]
[1] 11.125 -74.028
```

```
[[3]]
[1] 11.1302 -74.0215
```

3. Los guardabosques acaban de reportar un cuarto punto (11.1355, -74.0150). Utiliza el método nativo de tu lenguaje para agregar esta nueva coordenada al final de tu ruta.

0.1.5 Python

```
ruta_avistamiento.append((11.1355, -74.0150)) # Usa el método append para insertar al final de
```

0.1.6 R

```
ruta_avistamiento[[4]] <- c(11.1355, -74.0150) # Asigna el nuevo vector a la cuarta posición de
```

4. Utilizando la indexación dinámica (indexación negativa o comandos como length/end), extrae el último punto de la ruta y guárdalo en una variable llamada ultimo_reporte

0.1.7 Python

```
ultimo_reporte = ruta_avistamiento[-1] # Usa el índice -1 para acceder al último elemento
```

0.1.8 R

```
ultimo_reporte <- ruta_avistamiento[[length(ruta_avistamiento)]] # Usa length() para hallar el
```

5. Crea una nueva lista/vector paralela llamada elevaciones que contenga las alturas de los cuatro puntos: [850.5, 920.0, 1050.2, 1180.8].

0.1.9 Python

```
elevaciones = [ # Define una lista con valores de altura (float)
    (850.5),    # Altura punto 1
    (920.0),    # Altura punto 2
    (1050.2),   # Altura punto 3
    (1180.8)    # Altura punto 4
]
```

0.1.10 R

```
elevaciones <- c(850.5, 920.0, 1050.2, 1180.8) # Crea un vector numérico con las alturas
```

6. Calcula e imprime la elevación máxima alcanzada en el recorrido y el número total de puntos visitados (vértices), utilizando las funciones nativas (ej. max(), len(), length()).

0.1.11 Python

```
elev_maxima = max(elevaciones)          # Encuentra el valor más alto en la lista de elevaciones
puntos_visitados = len(ruta_avistamiento) # Cuenta el número total de elementos en la ruta
print(f"Elevación máxima: {elev_maxima} metros") # Imprime la elevación máxima formateada
```

Elevación máxima: 1180.8 metros

```
print(f"Total de puntos visitados: {puntos_visitados}") # Imprime el conteo de puntos
```

Total de puntos visitados: 4

0.1.12 R

```
elev_maxima <- max(elevaciones)          # Calcula el máximo valor del vector elevaciones
puntos_visitados <- length(ruta_avistamiento) # Obtiene el tamaño total de la lista
cat(paste("Elevación máxima:", elev_maxima, "metros\n")) # Imprime mensaje concatenado
```

Elevación máxima: 1180.8 metros

```
cat(paste("Total de puntos visitados:", puntos_visitados, "\n")) # Imprime mensaje concatenado
```

Total de puntos visitados: 4

0.2 Ejercicio 2

1. Crea un Diccionario (o Lista nombrada en R) llamado `pnn_tayrona` que contenga exactamente las siguientes claves y valores: “nombre”: “PNN Tayrona” “area_hectareas”: 15000 “abierto_turismo”: Verdadero (tipo Booleano) “ecosistemas”: Una lista/vector con los textos “Manglar”, “Bosque Seco”, y “Coral”.

0.2.1 Python

```
pnn_tayrona = {
    "nombre": "PNN Tayrona",          # Inicia el diccionario de Python
    "area_hectareas": 15000,          # Par clave-valor de tipo string
    "abierto_turismo": True,          # Par clave-valor de tipo entero
    "ecosistemas": ["Manglar", "Coral", "Matorral"] # Par clave-valor de tipo booleano
}
```

0.2.2 R

```
pnn_tayrona <- list(                                # Inicia una lista con nombres en R
  nombre = "PNN Tayrona",                          # Asigna valor al nombre
  area_hectareas = 15000,                          # Asigna valor numérico al área
  abierto_turismo = TRUE,                          # Asigna valor lógico al estado
  ecosistemas = c("Manglar", "Coral", "Matorral") # Asigna vector de texto a ecosistemas
)
```

2. Emplea la función o método de acceso seguro (ej. `.get()` en Python o condicionales en R/Julia) para intentar extraer la clave “fecha_creacion”. Como no existe, debe devolver el texto “Dato no disponible” sin que el programa colapse. Imprime el resultado.

0.2.3 Python

```
# .get() busca la clave; si no existe, devuelve el segundo argumento (valor por defecto)
resultado_fecha = pnn_tayrona.get("fecha_creacion", "Dato no disponible")

# Imprimimos el resultado
print(resultado_fecha) # Muestra "Dato no disponible" en lugar de un error
```

Dato no disponible

0.2.4 R

```
# Evalúa si la clave no existe (es nula) y asigna un valor por defecto
resultado_fecha <- if (!is.null(pnn_tayrona$fecha_creacion)) {
  pnn_tayrona$fecha_creacion # Si existe, extrae el valor
} else {
  "Dato no disponible"      # Si es NULL, devuelve este texto
}

print(resultado_fecha) # Muestra el resultado de la validación
```

[1] "Dato no disponible"

3. Actualiza el diccionario añadiendo una nueva clave llamada “departamento” con el valor “Magdalena”

0.2.5 Python

```
pnn_tayrona["departamento"] = "Magdalena" # Inserta una nueva clave con su valor respectivo
```

0.2.6 R

```
pnn_tayrona$departamento <- "Magdalena" # Crea dinámicamente un nuevo elemento en la lista
```

4. Recibes un reporte de campo crudo con los ecosistemas observados hoy, el cual contiene duplicados: `observaciones_crudas = ["Manglar", "Coral", "Bosque Seco", "Manglar", "Coral", "Matorral"]` Convierte esta colección en un Conjunto (Set o usando `unique()`) para eliminar los duplicados y guárdalo en la variable `observaciones_unicas`.

0.2.7 Python

```
observaciones_crudas = ["Manglar", "Coral", "Bosque Seco", "Manglar", "Coral", "Matorral"] # Lista de ecosistemas observados
observaciones_unicas = set(observaciones_crudas) # Convierte a set (conjunto) para purgar duplicados
print(f"Ecosistemas unicos detectados {observaciones_unicas}") # Muestra los elementos únicos
```

```
Ecosistemas unicos detectados {'Coral', 'Matorral', 'Bosque Seco', 'Manglar'}
```

0.2.8 R

```
observaciones_crudas <- c("Manglar", "Coral", "Bosque Seco", "Manglar", "Coral", "Matorral") # Lista de ecosistemas observados
observaciones_unicas <- unique(observaciones_crudas) # Filtra el vector dejando solo valores únicos
cat(paste("Ecosistemas únicos detectados:", paste(observaciones_unicas, collapse = ", "), "\n"))
```

```
Ecosistemas únicos detectados: Manglar, Coral, Bosque Seco, Matorral
```

5. Utiliza una operación lógica de conjuntos (intersección) entre la lista oficial de ecosistemas del parque (guardada dentro de tu diccionario en el paso 1) y tus `observaciones_unicas`. Imprime el resultado para saber qué ecosistemas oficiales fueron efectivamente avistados hoy.

0.2.9 Python

```
ecosistemas_oficiales = set(pnn_tayrona["ecosistemas"]) # Transforma la lista del diccionario a un set
# Encuentra los elementos presentes en ambos conjuntos (intersección)
avistamientos_confirmados = ecosistemas_oficiales.intersection(observaciones_unicas)
print(f"Los ecosistemas oficiales avistados hoy son: {avistamientos_confirmados}") # Muestra el resultado
```

```
Los ecosistemas oficiales avistados hoy son: {'Coral', 'Matorral', 'Manglar'}
```

0.2.10 R

```
ecosistemas_oficiales <- pnn_tayrona$ecosistemas # Extrae el vector del diccionario
# Encuentra los elementos comunes entre el vector oficial y las observaciones únicas
avistamientos_confirmados <- intersect(ecosistemas_oficiales, observaciones_unicas)

cat(paste("Los ecosistemas oficiales avistados hoy son:", paste(avistamientos_confirmados, coll
```

Los ecosistemas oficiales avistados hoy son: Manglar, Coral, Matorral

0.3 Pregunta

1. Topología Espacial y Seguridad: Tuplas vs. Listas En el desarrollo de software geoespacial, un punto representa una unidad atómica e indivisible. Inmutabilidad (Tupla): Al guardar (X, Y) como una tupla, garantizas la integridad referencial. Un punto geográfico no puede cambiar ni un poco; si la ubicación cambia, se trata de un nuevo punto. Esto evita que una función secundaria modifique accidentalmente una coordenada mientras realiza un cálculo.

Mutabilidad (Lista): La ruta es una entidad dinámica. Un trayecto crece, se recorta o se invierte. Una lista permite agregar (append) o eliminar puntos sin necesidad de recrear toda la estructura en memoria.

¿Qué pasa si intentas modificar `punto_1[0]`? Tanto en Python como en Julia (si defines el punto como una Tuple), el sistema lanzará un error de tipo (TypeError). Python: `TypeError: 'tuple' object does not support item assignment`. Julia: `MethodError: no method matching setindex!::Tuple{Float64, Float64}, ...`. Esto es una medida de seguridad: el lenguaje “protege” al dato de ser corrompido.

2. El dilema en R: `c()` vs. `list()` Para el campo `ecosistemas`, la opción técnica correcta es `c()` (vector atómico).

¿Por qué usar `c()`?

Homogeneidad: Todos los nombres de los ecosistemas son cadenas de texto (character). En R, los vectores atómicos son más eficientes en memoria y más rápidos para operaciones de conjunto (como `intersect` o `unique`).

Propósito: `list()` se usa en R para objetos heterogéneos (donde cada elemento puede ser de un tipo distinto: un número, un texto y otro vector). Como los ecosistemas son todos texto, `c("Manglar", "Coral")` es la forma idiomática y estándar.

3. La diferencia crítica en Slicing: `[1:3]` Aquí es donde ocurren la mayoría de los errores al saltar de un lenguaje a otro.

En Python: `ruta[1:3]` Resultado: Devuelve 2 elementos (`ruta[1]` y `ruta[2]`).

Lógica: Python usa Indexación Base-0 y el límite superior es excluyente. El slice `1:3` significa: “empieza en el índice 1 y detente antes del 3”.

En R y Julia: `ruta[1:3]` Resultado: Devuelve 3 elementos (`ruta[1]`, `ruta[2]` y `ruta[3]`).

Lógica: Ambos usan Indexación Base-1 y el límite superior es incluyente. El rango 1:3 significa literalmente “del primero al tercero, inclusive”.